Assignment A6: Probing & Interpretability (optional)

Individual assignment • 20% individual-report component Released: 14 Jan 2027 Due: 21 Jan 2027, 23:59

Objective. An *optional* sixth exercise for interpretability-leaning students. Rather than submit predictions, you will *probe* a model and report what you find. This assignment is graded **on the findings only** — there is no hidden test set.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.*

Scope

Required core: this *entire assignment* is optional (for interpretability-leaning students). If you take it: complete **one** of the two tracks and the 1–2 page `a6_findings.pdf`.

Optional: doing the second track as well, or extending the layer/pooling analysis. More tracks = more evidence, not a requirement.

1. Tasks (pick one track)

1. **Representation probing.** Train a lightweight linear probe on the frozen hidden states of DistilBERT (from A3) to predict the `subgroup` attribute. Report probe accuracy per layer and discuss what it implies about what the representation encodes.
2. **Attribution probing.** Using a feature-attribution method (`Captum` or `Inseq`) on your A3 classifier, extract the input tokens most responsible for a sample of predictions, and do a short **lexical/syntactic** analysis (POS tags, negation, named entities) of what drives the model.

Runs on your laptop

Frozen-model forward passes + a linear probe (or attribution over a few hundred examples) run on a laptop CPU/`mps` in minutes. Reuse the A3 checkpoint; no new training of the base model.

Deliverable & auto-grading

Submit `a6_findings.pdf` (max 2 pages) with: your method, 2–3 figures/tables, and a clear claim about what the model represents or relies on. **No prediction file; no auto-grading.** Graded on soundness of method, evidence, and the honesty of the claims (including negative results).

Pin these to avoid leakage. Fit the probe on `train` and evaluate on the disjoint `validation` split. Use **masked** mean-pooling (exclude padding). Report the **majority-class baseline** next to each probe accuracy. DistilBERT exposes **7** hidden-state tensors: index 0 = embeddings, 1–6 = transformer blocks.

2. Findings prompt

What does your model actually rely on — and is that what you would want it to rely on? Connect your evidence to fairness/robustness concerns where relevant.

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them*. Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A6_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.